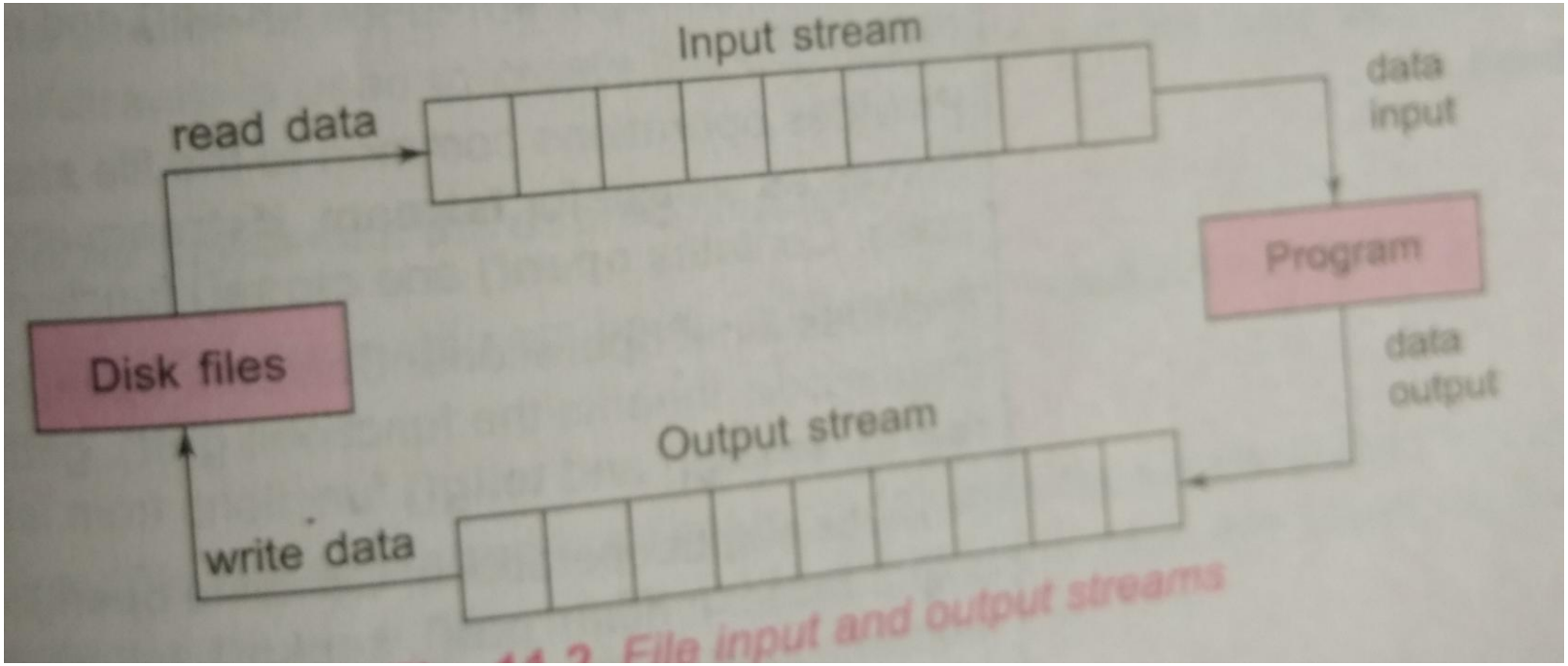
Why Files?

- Variables store data temporarily (in RAM).
- Files store data permanently on disk.
- Files are streams of data (input/output).

files

- The i/o system of C++ handles file operations which are similar to the console input and output operations
- It uses the streams as an interface between the programs and the files
- The stream that supplies data to the program is known as input stream and the one that receives data from the program is known as output stream
- Stream is a sequence of bytes



File input and output streams

files

- The I/O system of C++ contains a set of classes that defines the file handling methods
- These include ifstream, ofstream and fstream. These classes are derived form fstreambase and from the corresponding iostream classes
- These classes designed to manage the disk files are declared in fstream and therefore we must include this file in any program

File Stream Classes

- #include <fstream>
- ifstream – read from files.
- ofstream – write to files.
- fstream – read and write files.

files

- For opening a file we must create a file stream and then link to the file name
- Two ways to open file
  - Using the constructor
  - Member function open()

First method is useful when we use only one file in the stream. The second method is use when we want to manage multiple files using one stream

Writing to a File

- ofstream outFile('example.txt');
- Use << operator to write data.
- Always check if file opened successfully.
- Call close() after finishing.

Opening file

- ofstream outfile(“f1”);
- Ifstream infile(“f2”);
- You may perform input and output operations like

outfile<<“total”;

outfile<<“sum”;

Infile>>number;

The connection with file is closed automatically when the stream object expires(when program terminates)

outfile.close();

infile.close();

Opening a file — basic write example (line-by-line)

```
#include <iostream>
#include <fstream>   // file streams
using namespace std;

int main() {
   ofstream outFile("example.txt");  // 1. create/ofstream and open file for writing
   if (!outFile) {                   // 2. check whether file opened successfully
      cerr << "Error opening file for writing\n";
      return 1;
   }
   outFile << "Hello, world!\n";      // 3. write text to file using stream insertion
   outFile << 123 << "\n";           // 4. write numbers (formatted as text)
   outFile.close();                  // 5. close the file to flush buffers
   return 0;
}
```

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
ofstream outf("item");
cout<<"enter item name\n";
char name[30];
cin>>name;
outf<<name<<endl;
cout<<"enter item cost\n";
float cost;
cin>>cost;
outf<<cost<<endl;
outf.close();
ifstream inf("item");
inf>>name;
inf>>cost;
cout<<"\n";
cout<<"name= "<<name<<"cost=
"<<cost<<endl;
inf.close();
return 0;}
```

- ofstream outFile("example.txt"); — constructor opens file. Default mode is ios::out | ios::trunc which creates/truncates the file.

- if (!outFile) checks the stream's fail() state. Always do error checks.

- outFile << "..." uses the same operators as cout.

- close() flushes and releases resources. If you forget close(), destructor will close when outFile goes out of scope — but explicit close() is good practice in long functions.

- When we opened file for writing only, a new file is created if no file of that name
- If file exists then contents are deleted and the file is presented as a clean file

Reading from a File

- ifstream inFile('example.txt');
- Use getline(inFile, line) to read lines.
- Check if (!inFile) for errors.
- Close file after reading.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
   ifstream inFile("example.txt");   // open for reading
   if (!inFile) {
      cerr << "Error opening file for reading\n";
      return 1;
   }

   string line;
   while (getline(inFile, line)) {  // read line-by-line
      cout << line << '\n';
   }
   inFile.close();
   return 0;}
```

Opening file using open()

- It is used to open multiple files that use the same stream object

```
ofstream outfile;
outfile.open(“data1”);
…….
outfile.close();
  outfile.open(“data2”);
…..
outfile.close();
```

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   ofstream outFile;
   outFile.open("students.txt");
   if (!outFile) {cerr << "Error opening
file!" << endl;
      return 1;
   }
outFile << "Alice" << endl;
   outFile << "Bob" << endl;
   outFile << "Charlie" << endl;

   outFile.close
   cout << "Data written
successfully." << endl;
   return 0;
}
```

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
   ifstream inFile;
   inFile.open("students.txt");

   if (!inFile) {
      cerr << "File could not be
opened!" << endl;
      return 1;
   }
```

```
string name;
   while (getline(inFile, name)) {
      cout << name << endl;
   }

   inFile.close();
   return 0;
}
```

End of file

- Ifstrem object return 0 if any error occur in file operation including the end of file condition
- Another approach is
- fin.eof() it member function of ios class. It returns a non-zero value if EOF condition is encountered and zero otherwise
- In above program we can write
- while(!fin.eof())

File Modes

- ios::in – open for reading.
- ios::out – open for writing.
- ios::app – append at end.
- ios::ate go to end of file on opening
- ios::trunc – erase old contents.
- ios::binary – binary mode.
- ios::nocreate open fails if file dose not exists
- ios::noreplace open fails if file already exists

mode

- Ios::out mode also opens the file in ios::trunc mode by default
- Ios:app and ios::ate both take us to end of the file when it is opened. But app allows us to add data to the end of the file only while ate allows us to add or to modify the existing data anywhere in the file
- In both the mode file is created if does not exist. ios::app can be used only with the files capable of output
- When we used object of fstream then we must provide the mode
- The mode can be combined with or operator
- ios::app|ios::nocreate it fails if file does not exist

Append Example

- ofstream out('log.txt', ios::app);
- Adds data to end of file without erasing.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile;
    outFile.open("log.txt", ios::app);  // append mode

    if (!outFile) {
        cerr << "Could not open file!" << endl;
        return 1;
    }

    outFile << "New log entry." << endl;
    outFile.close();
}
```

Good Practices

- Always check for file open errors.
- Use while(getline(...)) for reading.
- Close files explicitly when done.

Common Errors

- File not found (path issue).
- File truncation when opened without app mode.
- Mixing text & binary data incorrectly.
- Forgetting to close file.

File Pointers

- seekg() – move read pointer.
- seekp() – move write pointer.
- tellg(), tellp() – get current positions.
- Used for random access in binary files.